

버스 ETA 신뢰도 데이터 / API 분석 보고서

서울시 vs TAGO · 1 시간 실측 PoC 기반

Prediction → Actual → Residual → Reliability

Prediction → Actual → Residual → Reliability

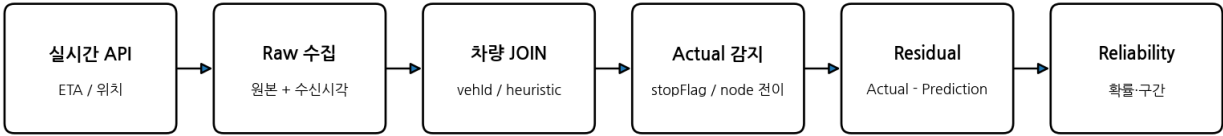


그림 1. 본 프로젝트의 핵심 데이터 흐름

서울 실험	753 번 / 59 분 45 초	240 cycles
TAGO 실험	대전 301 / 송강전통시장	240 cycles

작성 기준일: 2026-08-21

핵심 요약 - 아무것도 모르는 사람을 위한 2 분 설명

이 프로젝트는 무엇을 하는가?

버스 앱의 “5 분 후 도착”이라는 숫자를 다시 예측하는 프로젝트가 아니다. 그 5 분이라는 예측이 실제로 얼마나 자주 맞았고, 얼마나 흔들리는지를 과거 실측으로 누적해 “지금 이 ETA 를 어느 정도 믿어도 되는가”를 확률과 오차 범위로 보여주는 프로젝트다.

1. API가 제공하는 버스 도착예정시간(ETA)을 15 초 간격으로 반복 저장한다.
2. 같은 차량이 실제 정류장에 도착하는 순간을 위치 API에서 잡는다.
3. 예측시각과 실제 도착을 연결해 오차(Residual = Actual - Predicted)를 만든다.
4. 오차를 ETA 거리, 정류장, 시간대, 첫 번째/두 번째 도착예정버스 등의 조건으로 누적한다.
5. 사용자에게 “10 분 후”뿐 아니라 “이 예측은 ± 1 분 안에 들어올 확률이 약 몇 %”처럼 신뢰도를 제공한다.

핵심 질문	이번 실험의 답
예측 ETA 를 반복 수집할 수 있는가?	예. 서울 45,927 Prediction row, TAGO 181 성공 snapshot 을 축적했다.
같은 차량을 식별할 수 있는가?	서울은 vehId 로 직접 식별 가능. TAGO 도착 API 는 차량 ID 가 없어 위치정보와 heuristic 매칭이 필요하다.
실제 도착을 잡을 수 있는가?	서울은 stopFlag 전이와 source timestamp 로 interval Ground Truth 를 구성했다. TAGO 는 nodeord 전이로 후보 구간을 잡았다.
실제 ETA 신뢰도 패턴이 보이는가?	예. 서울에서 0~1 분 ETA 의 ± 60 초 이내 비율은 95.1 %, 10~20 분 ETA 는 37.5%로 크게 낮아졌다.
전체 노선으로 확장 가능한가?	기술적으로 가능하지만 개발계정 API 호출 한도가 가장 먼저 병목이 된다. 운영계정 증액·적응형 polling·route-wide 조회·분산 스트리밍이 필요하다.

가장 중요한 결론

이번 1 시간 데이터에서는 ETA 의 중앙 오차가 모든 구간에서 대체로 0 초 부근이었지만, ETA 가 멀어질수록 오차 분포의 폭이 크게 넓어졌다. 즉 “항상 몇 분 늦게 예측한다”보다 “멀리 있는 ETA 일수록 불확실성이 커진다”는 문제가 더 선명했다. 이는 Reliability Layer 의 필요성을 직접 지지한다.

1. 프로젝트 배경과 검증 목표

Handoff 문서가 정의한 원안의 핵심은 “더 정확한 ETA 를 새로 만드는 것”이 아니라 기존 ETA 의 오차를 누적해 신뢰도와 불확실성을 계량하는 것이다. 버스 파트에서는 API 목록을 많이 모으는 것이 아니라 다음 세 가지를 실제 response 로 증명하는 것을 성공조건으로 두었다.

- 특정 노선·정류장에 대한 예측 ETA 를 반복적으로 얻을 수 있는가?
- 동일 버스를 시간에 따라 식별하고 추적할 수 있는가?
- 특정 버스가 정류장에 실제 도착한 시각 또는 도착 구간을 잡을 수 있는가?

성공조건

한 차량에 대해 여러 prediction snapshot 과 하나의 actual arrival 을 연결하고, 각 lead time 별 residual 을 생성할 수 있어야 한다.

공통 엔진은 교통수단이 달라져도 다음 구조를 유지한다: Prediction → Actual → Residual → Reliability. 버스·지하철·고속열차의 시간 스케일이나 UI 임계값은 달라질 수 있지만 데이터 엔진의 핵심 상태 전이와 오차 누적 원리는 같다.

출처: 사용자 제공 Handoff: 「교통_신뢰도_서비스_버스_API_조사_Handoff(2).docx」

2. 어떤 API 를 조사했고, 각각 무엇을 주는가

Provider / API	실험 endpoint	핵심 입력	핵심 출력	본 프로젝트 역할
서울 도착정보	/arrive/getArrInfoByRouteAll	busRouteId	vehId1/2, exps1/2, mkTm, stId, staOrd	Prediction - 직접 차량 ID 포함
서울 위치정보	/buspos/getBusPosByRtid	busRouteId	vehId, plainNo, stopFlag,	Actual / vehicle state

			dataTm, sectOrd	
TAGO 도착정보	/ArvlInfo.../ getSttnAcctoArvlPrearngeInfoList	cityCode, nodeId	arrtime, arrprevstationcnt, routeid	Prediction - 차량 ID 없음
TAGO 위치정보	/BusLclInfo.../ getRouteAcctoBusLclList	cityCode, routeid	vehicleno, nodeord, nodeid, GPS	Vehicle state / Actual 후 보
TAGO 노선정보	/BusRouteInfo.../ getRouteAcctoThrhSttnList	cityCode, routeid	nodeid, nodeord, 경류장 명, 좌표	경적 route master / JOIN 보조

서울 도착 API의 핵심 장점은 한 번의 노선 조회가 그 노선의 전체 경유 정류소 도착예정정보를 반환한다는 점이다. 또한 Prediction 응답에 차량 ID(vehId1/vehId2)와 원천 생성시각(mkTm)이 함께 들어온다. 서울 위치 API는 같은 vehId를 반환하므로 차량 단위 direct JOIN이 가능하다.

TAGO 도착 API는 정류소(nodeId) 기준으로 현재 도착예정정보를 반환하며 ETA(arrtime)는 충분히 얻을 수 있지만, 도착 응답에 차량 ID와 source timestamp가 없다. 따라서 TAGO 위치 API의 vehicleno/nodeord와 결합할 때는 직접 JOIN이 아니라 heuristic 매칭이 필요하다.

3. 데이터는 실제로 어떻게 넘어오는가 - 실제 response로 보기

3.1 서울: Prediction response

실제 서울 도착 응답에서 핵심 필드만 발췌 (2026-08-21 14:17 경)

```
<itemList>
  <busRouteId>100100118</busRouteId>
  <stId>111000184</stId>
  <stNm>갈현동성당앞</stNm>
  <staOrd>5</staOrd>
  <mkTm>2026-08-21 14:17:20.0</mkTm>
  <vehId1>111033670</vehId1>
  <exps1>14</exps1>
  <sectOrd1>5</sectOrd1>
  <arrmsg1>곧 도착</arrmsg1>
</itemList>
```

필드	쉬운 의미
vehId1 = 111033670	“이 정류장에 첫 번째로 올 것으로 예측되는 차량”의 ID
exps1 = 14	원천 시스템이 제공한 ETA 14 초
mkTm = 14:17:20	이 Prediction 데이터가 원천에서 생성된 시각
stId / staOrd	예측 대상 정류장 ID와 노선상 순서

3.2 서울: 같은 차량의 Location response

동일 cycle의 실제 서울 위치 응답 발췌

```
<itemList>
  <vehId>111033670</vehId>
  <plainNo>서울 74 사 4520</plainNo>
  <dataTm>20260821141718</dataTm>
  <sectOrd>4</sectOrd>
  <stopFlag>1</stopFlag>
  <lastStnId>111000183</lastStnId>
  <nextStnId>111000136</nextStnId>
  <gpsX>126.915967</gpsX>
  <gpsY>37.619848</gpsY>
</itemList>
```

왜 이게 결정적인가?

Prediction 의 vehId=111033670 와 Location 의 vehId=111033670 가 동일하다. 즉 서울에서는 '이 ETA 가 어느 차량의 예측인가'를 추측하지 않고 차량 ID 로 직접 연결할 수 있다. 실제 240 cycle 중 238 cycle 에서 Prediction 과 Location 의 차량 ID 집합이 완전히 일치했고, 나머지 2 cycle 도 한쪽에 차량 1 대만 차이가 났다.

3.3 TAGO: Arrival 에는 ETA 는 있지만 차량 ID 가 없다

실제 TAGO 도착 응답 중 대전 301 번 항목

```
{
  "arrprevstationcnt": 6,
  "arrtime": 612,
  "nodeid": "DJB8001793",
  "nodenm": "송강전통시장",
  "routeid": "DJB30300054",
  "routeno": 301,
  "routetp": "간선버스",
  "vehicletp": "저상버스"
}
```

여기에는 arrtime=612 초, arrprevstationcnt=6 같은 유용한 Prediction 정보가 있지만 vehicle_id/vehiclno 가 없다. 따라서 다음 15 초 응답의 301 번이 같은 차량인지, 앞 차량이 도착한 뒤 뒤 차량으로 바뀐 것인지 response 만으로 확인할 수 없다.

3.4 TAGO: Location 에는 차량번호와 노선상 위치가 있다

실제 TAGO 위치 응답 중 차량 1 대 발체

```
{
  "gpslati": 36.442641,
  "gpslong": 127.384534,
  "nodeid": "DJB8001285",
  "nodenm": "대광로제비앙아파트",
  "nodeord": 3,
  "routenm": 301,
  "routetp": "간선버스",
  "vehiclno": "대전 75 자 9384"
}
```

Location 에는 vehiclno 와 nodeord 가 있으므로 차량을 시간에 따라 추적할 수 있다. 하지만 Arrival 에는 vehiclno 가 없기 때문에 TAGO 에서 Prediction→Vehicle 연결은 “남은 정류장 수(arrprevstationcnt) $\approx$ target_nodeord - current_nodeord”를 이용한 heuristic 이다.

4. 실제 한 차량에서 Prediction → Actual → Residual 이 만들어지는 과정

다음은 서울 753 번의 차량 111033034 가 “사천교” 정류장에 도착한 실제 event 다. 같은 차량·같은 정류장에 대해 여러 시점의 ETA 가 쌓였고, Location 의 stopFlag 전이로 Actual interval 을 잡았다.

실제 Location snapshot 전이

```
15:07:49 source | vehicle=111033034 | stopFlag=0 | section_ord=80
15:08:10 source | vehicle=111033034 | stopFlag=1 | section_ord=81 | last_stop_id=112000026 (사천교)
```

⇒ Actual arrival $\in$ (15:07:49, 15:08:10]
⇒ uncertainty width = 21 초

Prediction 생성	제공 ETA	환산 Predicted arrival	Residual midpoint
14:53:06	892 초	15:07:58	+1.5 초
14:58:57	598 초	15:08:55	-55.5 초
15:03:53	303 초	15:08:56	-56.5 초
15:07:09	65 초	15:08:14	-14.5 초

Residual의 부호

Residual = Actual - Predicted. 음수이면 실제 버스가 예측시각보다 일찍 도착했고, 양수이면 예측보다 늦게 도착했다는 뜻이다. Actual은 21 초 interval 이므로 midpoint는 대표값일 뿐이며, 분석에서는 interval의 lower/upper도 함께 보존했다.

5. 1 시간 수집 실험 - 얼마나 많은 데이터가 쌓였는가

Dataset	Rows / Calls	설명
서울 raw API	480 calls	Arrival 240 + Location 240; 모두 HTTP/서비스 정상
서울 Prediction	45,927 rows	20 vehicles / 103 target stops
서울 Location	3,601 rows	20 vehicles
서울 Actual candidate	438 events	102 stops; interval/first-observed 포함
서울 Residual	33,867 rows	여러 Prediction 이 하나의 Actual 에 연결되며 증폭
TAGO Prediction	181 success snapshots	240 cycle 중 성공한 301 번 Prediction
TAGO Location	4,238 rows	193 성공 cycle / 25 vehicles / 100 nodes
TAGO Actual	8 events	nodeord 94→95 target 진입
TAGO Residual	107 rows	heuristic match EXACT 69 + NEAR 38

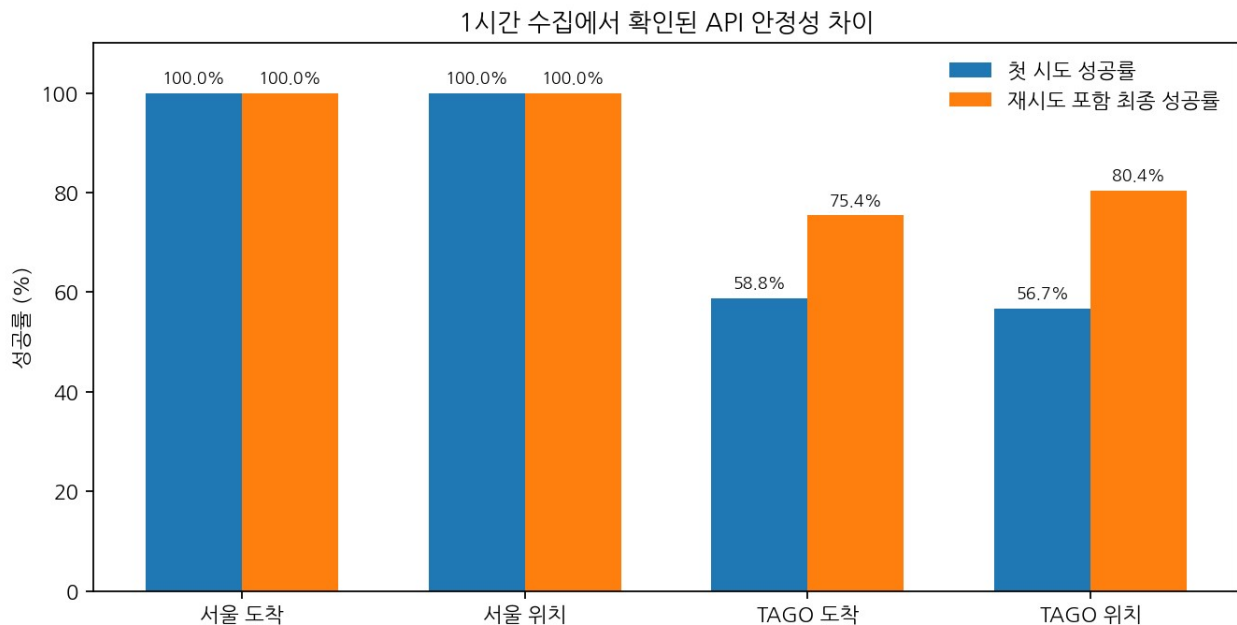


그림 2. 1 시간 동안의 API 성공률 - 서울은 480/480 정상, TAGO는 HTTP_ERROR 04로 반복 결측

TAGO는 HTTP status가 200 이어도 body가 OpenAPI_ServiceResponse/HTTP_ERROR 04인 경우가 반복됐다. 따라서 collector는 HTTP 200만 보고 성공 처리하면 안 되고 resultCode/body 구조를 반드시 검사해야 한다. 이번 240 cycle에서 첫 시도 성공률은 Arrival 58.8%, Location 56.7%였고, 1회 retry를 포함해도 각각 75.4%, 80.4%였다.

6. 데이터 품질 점검과 Cleaning

6.1 polling row ≠ 원천의 새로운 snapshot

항목	원본	source-level unique	반복 snapshot
서울 Prediction	45,927	39,247	6,680 (14.5%)
서울 Location	3,601	2,861	740 (20.5%)
서울 Residual	33,867	28,591	5,276 (15.6%)

15 초마다 요청했지만 원천 timestamp 가 그대로인 응답이 다시 제공되는 경우가 있었다. 서울 Arrival 의 unique source timestamp 갱신 간격 중앙값은 약 17 초, Location 의 차량별 unique 갱신 간격 중앙값은 약 19 초였다. 따라서 “collector 가 받은 횟수”를 독립 Prediction 수로 세면 표본 수가 부풀려진다. 원본은 보존하되 분석용 Silver layer 에서는 source-generated timestamp 와 핵심 필드를 기준으로 dedupe 했다.

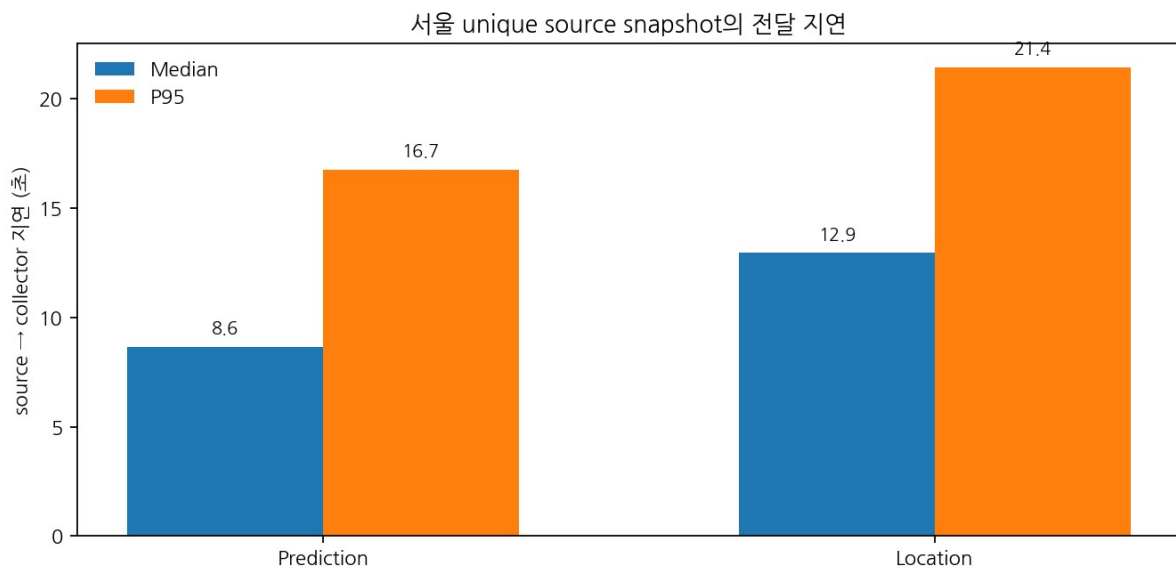


그림 3. 중복을 제거한 서울 source snapshot의 전달 지연

6.2 수신 시점에 이미 만료된 ETA

Prediction source dedupe 후 784 건(2.00%)은 predicted_arrival_at 이 collector_received_at 보다 이미 과거였다. 즉 사용자가 API 를 받은 순간에는 해당 ETA 시각이 지나간 경우다. Main 분석에서는 이를 ETA_EXPIRED_ON_RECEIPT 로 분리하고 대표 Reliability 계산에서는 제외했다.

6.3 Actual Ground Truth 는 단일 시각이 아니라 interval 로 관리

Grade	정의	건수	용도
A	stopFlag 0→1 직접 전이 + uncertainty ≤45 초	398	Main analysis
B	stopFlag 1→1 이지만 정류장 ID 가 변경 + uncertainty ≤45 초	28	Sensitivity / 보조
C	첫 snapshot 부터 stopFlag=1 또는 uncertainty >45 초	12	대표 분석 제외

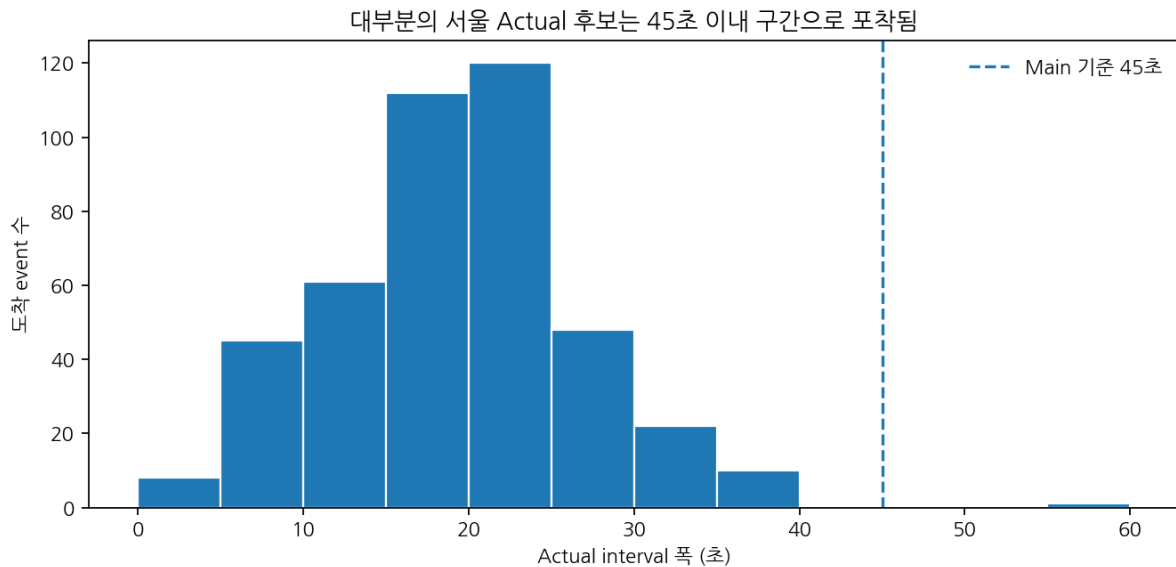


그림 4. 서울 Actual transition interval의 폭 - 428 bracket event 중 426 건이 45 초 이내

428 개의 bracket event 에서 uncertainty 중앙값은 18 초, P95 는 32 초였다. 최대 466 초 outlier 가 1 건 존재했다. 위치 snapshot 자체의 source timestamp 를 사용했기 때문에, 단순 collector 수신시각보다 Ground Truth 품질이 높다.

6.4 계산 무결성 검증

- 45,927 Prediction 전체에서 $\text{predicted_arrival_at} = \text{source_generated_at} + \text{eta_sec}$ 재계산 불일치 0 건.
- 33,867 Residual 전체에서 lower/upper/midpoint residual 재계산 불일치 0 건.
- Location 을 시간순 replay 해 Actual detector 를 다시 구현했을 때 438 event 가 기존 Actual CSV 와 438/438 일치.

7. 최종 Reliability 분석용 데이터셋

정제 단계	Residual rows	Actual events
원본 Residual	33,867	428 bracket events
동일 source Prediction 중복 제거	28,591	428
Actual Grade A 만	26,951	398
Prediction 이 Actual lower bound 전에 수신된 경우만	26,427	396
수신 시 이미 만료된 ETA 제외 - Main	26,253	395

왜 26,253 을 그냥 N=26,253 으로 보면 안 되는가?

395 개의 실제 도착 event 하나당 여러 Prediction snapshot 이 붙어 있다. Main 데이터에서 event 당 snapshot 수는 평균 66.5 개, 중앙값 72 개다. 따라서 snapshot 을 단순 평균하면 많이 관측된 event 가 결과를 지배한다. 대표 결과는 각 Actual event 가 동일한 총 weight 를 갖는 event-weighted 방식으로 계산했다.

전체 지표	Snapshot-weighted	Event-weighted (대표)
MAE	83.5 초	76.6 초
±30 초 이내	27.5%	32.7%
±60 초 이내	48.2%	54.1%
±120 초 이내	75.8%	79.6%

8. Reliability 분석 결과

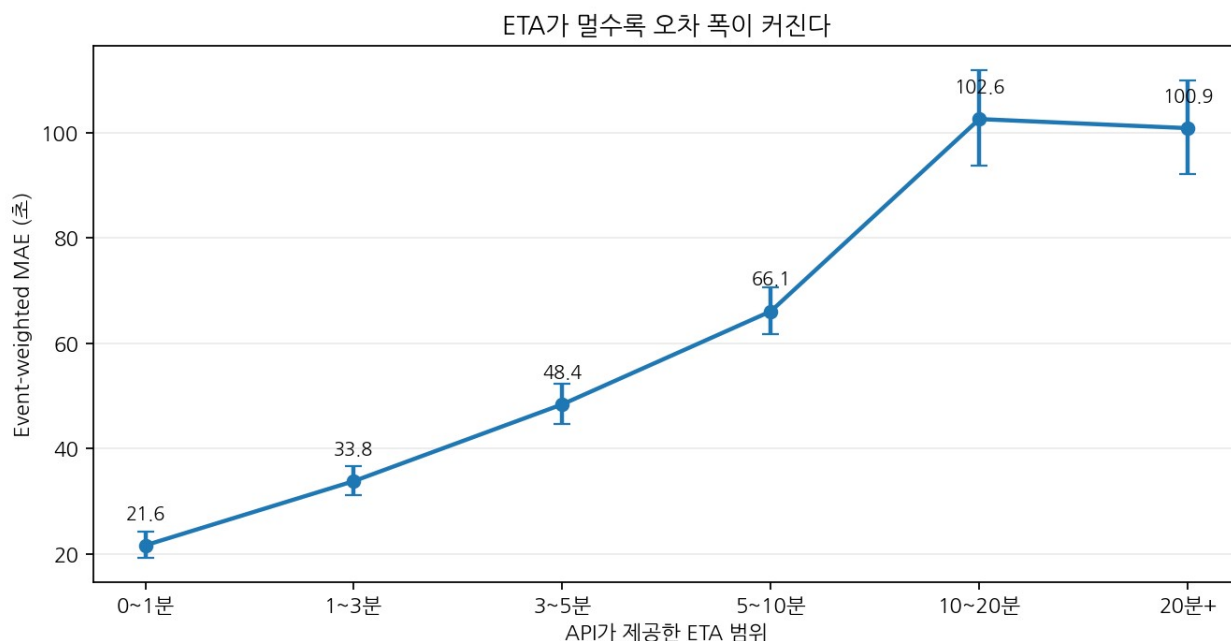


그림 5. ETA 범위별 Event-weighted MAE 와 Actual-event bootstrap 95% CI

ETA	Events	MAE	P10 residual	Median	P90 residual	±60 초
0~1 분	307	21.6 초	-23.0	3.5	40.0	95.1%
1~3 분	384	33.8 초	-53.0	-4.5	50.0	85.1%
3~5 분	358	48.4 초	-82.5	-3.5	77.0	67.2%
5~10 분	340	66.1 초	-106.5	3.5	106.0	52.2%
10~20 분	311	102.6 초	-167.5	-3.0	139.5	37.5%
20 분+	206	100.9 초	-162.5	-10.5	167.5	33.3%

핵심 패턴은 “bias 가 크게 한쪽으로 치우친다”보다 “ETA 가 멀수록 분산이 커진다”이다. 모든 bucket에서 residual 중앙값은 대체로 0 초 부근이지만 80% 범위(P10~P90)가 0~1 분에서는 약 -23~+40 초, 10~20 분에서는 약 -168~+140 초까지 넓어진다.

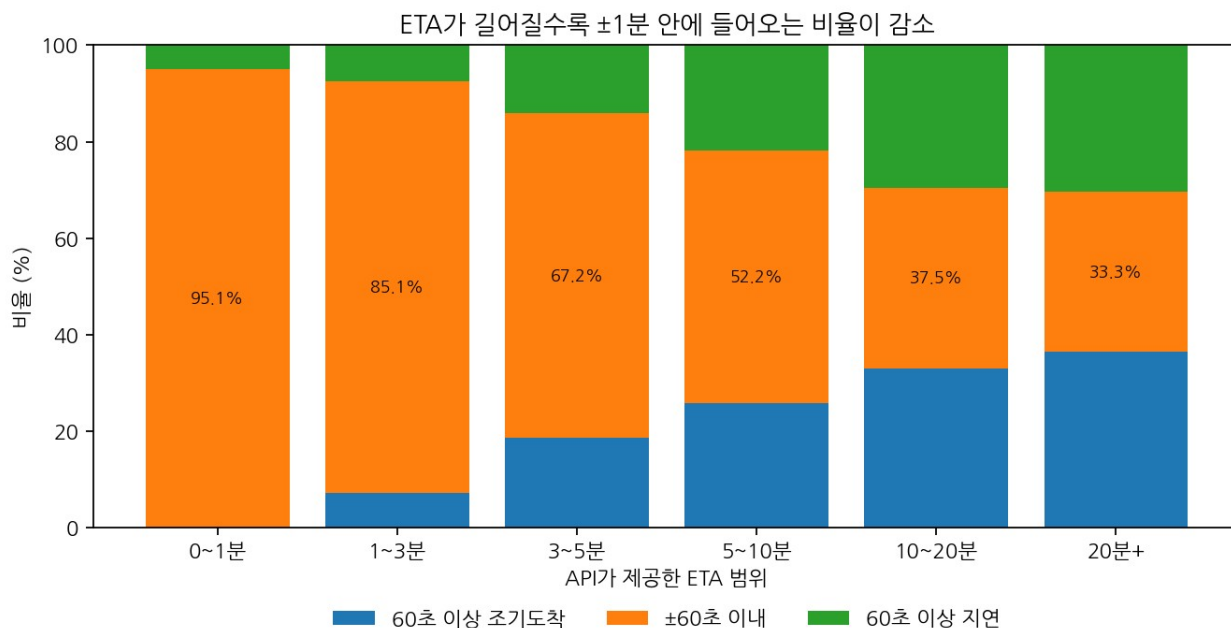


그림 6. ±60 초 관점의 조기도착 / 허용범위 / 지연 비율

사용자 관점으로 번역하면

0~1 분 남았다고 표시될 때는 약 95%가 ± 1 분 안에 들어왔지만, 10~20 분 남았다고 표시될 때는 약 37%만 ± 1 분 안에 들어왔다. 긴 ETA는 “틀렸다/맞았다” 이분법보다 “오차 범위가 넓다”는 신뢰도 정보가 함께 있어야 행동 판단에 도움이 된다.

8.1 Actual interval 때문에 만들어진 착시인가?

ETA	Actual interval 내 최소 가능 MAE	Midpoint MAE	최대 가능 MAE
0~1 분	14.2	21.6	30.6
1~3 분	25.7	33.8	43.0
3~5 분	39.9	48.4	57.7
5~10 분	57.3	66.1	75.4
10~20 분	93.7	102.6	111.9
20 분+	92.2	100.9	110.0

Actual interval 내부에서 가장 유리한 실제시각을 택해도, 가장 불리한 시각을 택해도 lead-time 에 따라 오차가 커지는 순서는 유지된다. Grade B 까지 포함한 Extended set 에서도 bucket MAE 변화는 약 1 초 수준에 그쳐 Main 결론은 품질 gate 선택에 민감하지 않았다.

9. 조건부 Reliability 에서 발견한 추가 패턴**9.1 첫 번째 버스와 두 번째 버스**

ETA	동일-event paired 수	Rank2 MAE - Rank1 MAE	95% bootstrap CI
5~10 분	41	+22.5 초	[13.6, 31.7]초
10~20 분	200	+11.9 초	[4.8, 19.0]초
20 분+	21	+18.1 초	[-8.8, 47.2]초

동일 Actual event 와 동일 ETA bucket 안에서 rank1/rank2 가 모두 존재한 경우만 비교했다. 5~10 분, 10~20 분에서 두 번째 도착예정버스(rank2)의 오차가 유의하게 더 컸다. 따라서 향후 Reliability 모델은 lead time 뿐 아니라 arrival_rank 를 최소 조건 변수 후보로 가져갈 가치가 있다.

9.2 stop order 51~60 × rank2 anomaly

구간 / Rank	Events	MAE	Bias	± 60 초
51~60 / Rank1	40	45.5 초	-12.7 초	77.0%
51~60 / Rank2	31	234.8 초	-171.4 초	11.5%

이 구간은 rank2 에서만 매우 큰 조기도착 방향 오차가 관측됐다. 노선 순서상 비슷한 지역 정류장이 반대 순서로 다시 나타나는 구간과 겹치지만, 이것이 회차/방향 전환 때문이라고 단정할 증거는 없다. “강한 anomaly 관측, 원인 미검증”으로 분류하고 다일·다노선 수집에서 재현 여부를 확인해야 한다.

9.3 지금 말하면 안 되는 것

- 정류장별 Reliability 를 확정 수치로 발표: 대부분 정류장 Grade A event 가 약 4 건이라 표본이 너무 작다.
- 차량별 성능 차이를 차량 자체 특성으로 해석: 각 차량이 경험한 노선구간·lead time·rank 분포가 다르다.
- 서울 전체 버스의 일반적 정확도라고 표현: 이번 데이터는 753 번 한 노선, 약 1 시간의 PoC 다.

10. 서울 vs TAGO - 최종 데이터 소스 판정

항목	서울	TAGO
Prediction 차량 ID	vehId1/vehId2 - 직접 제공	도착 API 에는 없음
Location 차량 ID	vehId	vehicleno
Prediction↔Location	Direct ID JOIN	arrprevstationcnt↔nodeord heuristic
Prediction source timestamp	mkTm 제공	없음
Location source timestamp	dataTm 제공	없음
Actual signal	stopFlag + stop ID 전이	nodeord target 진입/crossing
1 시간 수집 안정성	480/480 정상	04 오류 다수; retry 후에도 결측
Reliability 주 분석 적합성	Primary	Secondary feasibility / 범용화 비교

판정

MVP 의 Prediction→Actual 을 정확하게 증명하는 목적에는 서울 API 가 명백히 우위다. TAGO 는 전국 범용성 측면에서 가치가 있지만 vehicle identity·source timestamp·API 안정성 문제 때문에 현재는 동일 수준의 Ground Truth 로 설치 않는다.

TAGO residual 107 건에서 큰 음수 bias 가 보이지만 이를 서울과 비교해 “TAGO ETA 가 더 부정확하다”고 결론내리면 안 된다. TAGO 의 predicted_arrival_at 은 source generated time 이 없어 collector_received_at + arrtime 으로 재구성한 값이고, vehicle join 도 heuristic 이기 때문에 provider accuracy 비교의 전제가 다르다.

11. API 호출 제한과 전체 노선 확장 전략

11.1 공식 기본 트래픽

API product	공공데이터포털 개발계정 표시	운영계정
서울 버스도착정보	1,000	활용사례 등록 후 트래픽 증가 신청 가능
서울 버스위치정보	1,000	활용사례 등록 후 트래픽 증가 신청 가능
TAGO 버스도착정보	10,000	활용사례 등록 후 트래픽 증가 신청 가능
TAGO 버스위치정보	10,000	활용사례 등록 후 트래픽 증가 신청 가능
TAGO 버스노선정보	10,000	정적 master 이므로 실시간 polling 불필요

출처: 공공데이터포털 공식 페이지(2026-08-21 확인): data.go.kr/data/15000314, 15000332, 15098530, 15098533, 15098529. “신청 가능 트래픽”은 계정의 일일 호출 예산으로 운영되는 공공데이터포털의 일반적 트래픽 개념이며 실제 프로젝트 키의 승인/증액치는 활용신청 화면에서 최종 확인해야 한다.

11.2 15 초 polling 을 그대로 전체 노선에 적용하면?

Polling interval	1 route × 1 endpoint / day	Arrival+Location / route / day
15 초	5,760	11,520
30 초	2,880	5,760
60 초	1,440	2,880
90 초	960	1,920
120 초	720	1,440

서울 개발계정 1,000 이라는 기본 traffic 으로는 15 초 수집(5,760 calls/day/endpoint)은 한 노선도 지속할 수 없다. 단일 노선을 24 시간 계속 돌리려면 이론상 약 86.4 초보다 긴 평균 주기가 필요하므로 90 초 polling 에서 960 calls/day 수준이 된다. 그러나 90 초는 Actual interval 을 지나치게 넓히므로 학습·검증용 고품질 수집에는 부적합하다.

따라서 “전체 노선 = polling 간격만 늘린다”는 해법은 아니다.

실시간 Reliability 를 유지하면서 전체 노선을 확장하려면 운영계정 트래픽 증액 또는 원천기관과의 별도 대량 제공 경로가 우선이다. 호출 한도를 피하기 위해 다계정/다키를 우회 사용하는 방식은 설계 대안으로 두지 않는다.

11.3 이미 묶어서 받고 있는 부분

- 서울 Arrival: getArrInfoByRouteAll 은 한 route ID 의 “전체 경유 정류소” 도착예정정보를 한 번에 반환한다. 현재 방식 자체 가 정류장별 호출보다 훨씬 효율적이다.
- 서울 Location: 실제 사용한 getBusPosByRtid 는 한 route ID 의 운행 차량 위치 목록을 한 번에 반환한다. 서울 열린데이터광 장 공식 설명에도 동일 endpoint 예시가 존재한다.
- TAGO Location: getRouteAcctoBusLcList 도 한 route ID 의 차량 목록을 한 번에 반환한다.
- TAGO Arrival: 테스트한 operation 은 nodeId 기준이다. 한 호출이 해당 정류장의 여러 노선을 반환하지만, 한 노선의 모든 정류장 ETA 를 얻기 위한 route-wide arrival operation 으로 쓰기는 어렵다.
- 노선/정류장 master: 서울은 월별 XLSX, TAGO 는 Route API 를 시작 시 1 회 또는 일 단위 캐시로 사용해 실시간 트래픽을 소 비하지 않는다.

11.4 호출량을 줄이면서 정보 손실을 최소화하는 전략

전략	구체안	효과 / 주의
운영계정 증액	활용사례 등록 후 서울/TAGO traffic 증액 신청	전체 노선 실시간화의 선행조건
Adaptive polling	평상시 30~60 초, 도착 임박/관심 노선은 10~15 초로 승격	이번 데이터의 source 갱신 중앙값 17~19 초를 활용; 동일 snapshot 반복 수집 감소
Route cohort 수집	전 노선을 동시에 24 시간이 아니라 노선군·시간 대를 순환	학습 데이터 확보 단계에서 quota 효율 증가
Peak window 집중	출근/낮/퇴근/주말 등 대표 window 에 고해상도 수집	시간대 조건별 Reliability 를 확보하면서 호출 예 산 통제
Static master 분리	노선/정류장 정보는 월별 파일 또는 cache	실시간 API 를 Prediction/Location 에 집중
Provider bulk/data request	운영 단계에서 서울/TOPIs 등에 대량·연계 제공 가능성 협의	수백 노선 24/7 에서 가장 현실적인 경로

12. 데이터가 부족할 때의 “증강”은 어떻게 해야 하는가

Reliability 는 실제 Prediction 과 실제 Actual 의 관계를 측정하는 것이므로 가짜 Ground Truth 를 만들어 표본을 늘리는 방식은 대표 성능 계산에 사용하면 안 된다. 대신 다음과 같이 “실측을 보존하면서 희소성을 완화”해야 한다.

방법	사용 방식	대표 Reliability 에 사용?
다일 반복 수집	같은 route/stop 을 평일·주말·시간대별로 누 적	예 - 가장 우선
계층적 pooling / shrinkage	표본이 적은 정류장은 노선·시간대·lead bucket 의 상위 분포를 prior 로 사용	예 - 불확실성 명시
Bootstrap	Actual event 단위 재표집으로 CI 계산	예 - 통계 안정성 측정
과거 운행/구간 평균 데이터	평균 운행시간, 배차, 교통량 등을 조건 변수로 결함	예 - feature 로 사용; 과거 ETA 대체는 불가
Synthetic / simulation	부하테스트·Flink state stress·장애 시나리오 생성	아니오 - Ground Truth 평가와 분리

과거 ETA 의 한계

테스트한 실시간 도착 API 는 과거 시점을 넣어 예전 Prediction snapshot 을 재조회하는 기능이 없다. 서울 열린데이터광 장 FAQ 도 실시간 데이터는 대용량·분단위 제공 때문에 과거 데이터를 별도 적재하지 않는 경우가 있다고 안내한다. 따라서 “오늘부 터 쌓는 raw snapshot 이 미래의 학습 데이터”가 된다.

13. 전체 노선에서는 왜 분산 스트리밍이 필요한가

한 노선 1 시간은 Python 한 프로세스로도 처리 가능하다. 분산처리를 정당화하는 핵심은 “파일이 몇 GB 라서”가 아니라, 전체 노선 으로 확장될 때 발생하는 지속 스트림·수많은 keyed state·fault tolerance·replay 요구다.

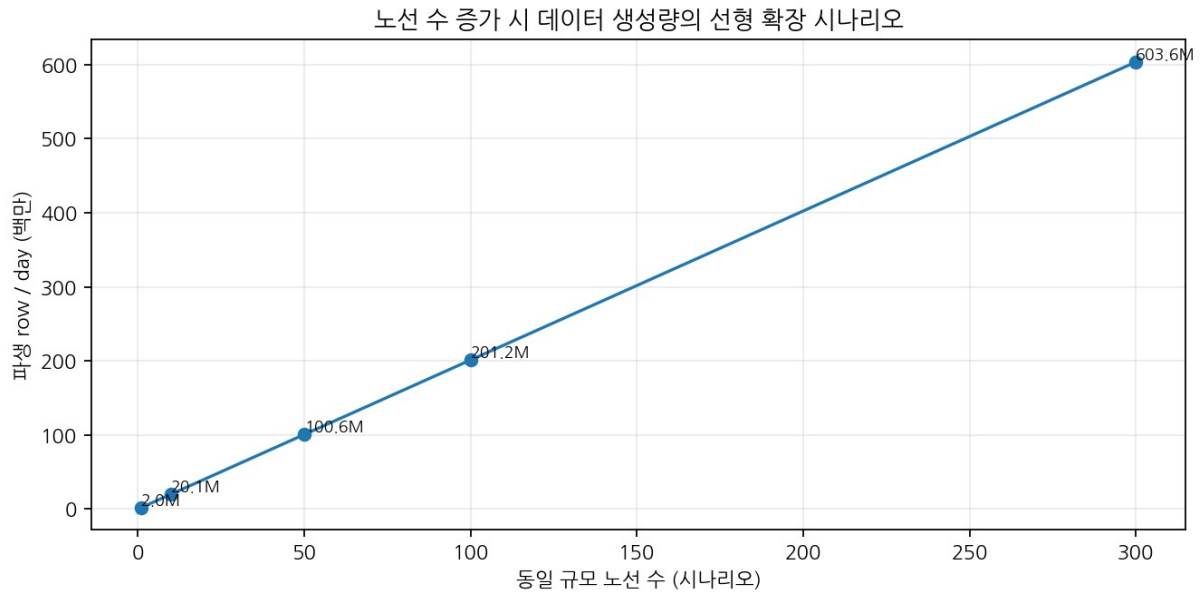


그림 7. 753 번과 비슷한 데이터 생성량을 가정한 단순 선형 scale-order 시나리오 (서울 전체 실제량 추정치가 아님)

실험에서 관측된 규모	값
서울 파생 rows / hour	Prediction 45,927 + Location 3,601 + Actual 438 + Residual 33,867 = 83,833
서울+TAGO 파일 footprint	약 76.2 MiB / 1 시간
서울 raw XML	약 56.4 MiB / 1 시간 → gzip level 6 약 3.26 MiB
서울 한 노선 active key 구조	(route, vehicle, target_stop)에 Prediction state 가 누적되고 Actual 에서 flush

Kafka/Flink 가 자연스러운 이유

Collector 와 계산을 분리하고 raw event 를 Kafka 에 보존하면 Flink 장애 후 replay 가 가능하다. Flink 는 route/vehicle/stop key 별 Prediction state 와 vehicle state 를 병렬로 유지하고 stopFlag 전이에서 Residual 을 emit 할 수 있다. 전 노선에서는 이 state 가 자연스럽게 partition 되어 수평 확장된다.

Prediction → Actual → Residual → Reliability

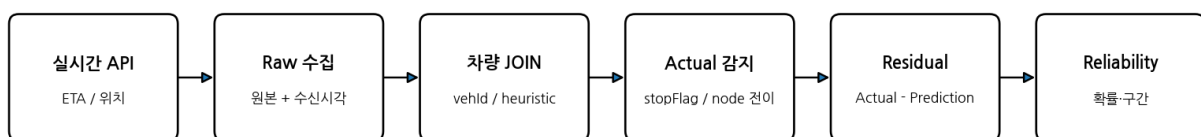


그림 8. PoC Python collector 에서 검증한 상태 흐름을 Kafka/Flink 로 확장

Layer	저장/처리 내용
Bronze	원본 XML/JSON, HTTP/body error, 요청/수신시각 - 수정하지 않고 replay 가능하게 보존
Silver	정규화, source timestamp, source-level dedupe, route master JOIN, quality flags
Gold	Actual interval, Residual event, lead-time/노선/정류장/시간대 Reliability 집계

14. 현재 한계와 위험

위험	왜 문제인가	대응
1 시간·1 노선 표본	시간대/노선 전체로 일반화 불가	다일·복수 노선 수집, 출퇴근/주말 계층화
Actual 은 interval	정확한 문 열림 시각이 아님	lower/upper 보존, 45 초 quality gate, interval sensitivity
실시간 API quota	전체 노선 고빈도 polling 의 1 차 병목	운영계정 증액 + adaptive polling + bulk 제공 협의
TAGO 04 오류	연속 결측과 retry 비용	body 기반 성공판정, backoff, quality stream
TAGO vehicle join heuristic	잘못된 Prediction→Actual 연결 가능	서울을 Primary, TAGO 는 confidence/quality 분리
source 반복 snapshot	독립 표본 수 과대계상	source timestamp 기반 dedupe
schema/endpoint 변경	실시간 파이프라인 깨질 수 있음	Bronze raw 보존 + schema version + replay 테스트

15. 최종 제언 - 다음 수집/개발 우선순위

1. 서울을 Main bus source 로 확정한다. vehId direct JOIN, mkTm/dataTm, stopFlag 때문에 Prediction→Actual 의 증거 품질이 가장 높다.
2. 현재 753 번 collector 를 3~7 일 이상, 출근·낮·퇴근·주말 window 로 확장해 lead-time Reliability 를 안정화한다.
3. 2~5 개의 성격이 다른 노선으로 확장해 51~60 구간 anomaly 와 rank2 효과가 재현되는지 검증한다.
4. 운영계정 traffic 증액을 즉시 신청한다. 전체 노선 실시간 PoC 의 feasibility 는 API quota 에 직접 좌우된다.
5. Collector 출력 형식을 Bronze/Silver event schema 로 고정하고 Kafka replay 가 가능한 JSON/Avro/Protobuf schema 를 정의한다.
6. Flink 에서는 Location 을 keyBy(route_id, vehicle_id), Prediction state 를 논리적으로 (route_id, vehicle_id, target_stop_id) 단위로 관리하고 TTL 을 둔다.
7. TAGO 는 전국 확장 가능성 비교 소스로 유지하되, residual 에는 join_method / join_quality / actual_method 를 반드시 기록한다.

보고서 전체 결론

“예측 ETA 자체를 새로 만드는 것”보다 “제공된 ETA 가 현재 얼마나 믿을 만한지 계량한다”는 서비스 방향은 실제 버스 데이터에서 end-to-end 로 구현 가능했다. 서울 753 번 1 시간 PoC 에서는 ETA 가 멀어질수록 오차가 뚜렷하게 커졌고, 차량 ID 기반 Prediction→Location JOIN 과 stopFlag 기반 Actual interval 을 통해 Reliability 계산의 기반을 확보했다. 다음 병목은 알고리즘보다 데이터 기간·노선 다양성·API traffic 이다.

Appendix A. 공통 이벤트 스키마 권장안

필드	의미	서울	TAGO
source_mode	BUS / SUBWAY / HIGH_SPEED_RAIL	BUS	BUS
source_provider	원천 제공자	SEOUL	TAGO
route_id	노선 ID	busRouteId	routeid/routeld
stop_id	정류장 ID	stId / lastStnId	nodeid
vehicle_or_trip_id	차량/편 식별자	vehId	Prediction 없음 / Location vehicleno
prediction_received_at	collector 수신시각	가능	가능
prediction_generated_at	원천 Prediction 생성시각	mkTm	미제공
predicted_eta_sec	상대 ETA	exps1/2	arrtime
predicted_arrival_at	환산 예측 도착시각	mkTm + exps	received_at + arrtime (재구성 표시 필요)
actual_lower/upper	Actual interval	stopFlag transition	nodeord crossing
residual_lower/upper	Actual-Prediction 범위	가능	heuristic 품질 플래그 필요
join_method / quality	연결 근거	VEHICLE_ID_DIRECT	TARGET_GAP_HEURISTIC
quality_flag	stale/error/interval 품질	필수	필수

Appendix B. 사용 데이터와 공식 출처

B.1 사용자 제공 실측 파일

- seoul_raw_20260821_141735.jsonl
- seoul_predictions_20260821_141735.csv
- seoul_locations_20260821_141735.csv
- seoul_actual_candidates_20260821_141735.csv
- seoul_residual_candidates_20260821_141735.csv
- tago_raw_20260821_141244.jsonl
- tago_prediction_snapshots_20260821_141244.csv
- tago_vehicle_locations_20260821_141244.csv
- tago_actual_candidates_20260821_141244.csv
- tago_residual_candidates_20260821_141244.csv
- 교통_신뢰도_서비스_버스_API_조사_Handoff(2).docx
- 지하철_연착_확률_메트릭(3).pdf

B.2 공식 API / 데이터 페이지 (2026-08-21 확인)

서울특별시_버스도착정보조회 서비스

<https://www.data.go.kr/data/15000314/openapi.do>

서울특별시_버스위치정보조회 서비스

<https://www.data.go.kr/data/15000332/openapi.do>

서울시 노선버스위치 목록정보 - getBusPosByRtid 예시

<https://data.seoul.go.kr/dataList/OA-15336/L/1/datasetView.do>

서울시 버스 노선 정보 / 월별 정류소 master

<https://data.seoul.go.kr/dataList/OA-1095/L/1/datasetView.do>

서울시 버스운행노선 월별 파일

<https://data.seoul.go.kr/dataList/OA-15066/F/1/datasetView.do>

TAGO 버스도착정보

<https://www.data.go.kr/data/15098530/openapi.do>

TAGO 버스위치정보

<https://www.data.go.kr/data/15098533/openapi.do>

TAGO 버스노선정보

<https://www.data.go.kr/data/15098529/openapi.do>

서울 열린데이터광장 - 실시간 데이터 과거값 FAQ 참고

<https://data.seoul.go.kr/dataList/datasetView.do?infId=OA-12601&serviceKind=1&srvType=A>

라이선스/출처표시: 서울특별시 공개데이터는 해당 페이지의 공공누리/출처표시 조건을 따르고, TAGO 는 각 공공데이터포털 이용허락범위를 따른다. 본 보고서의 response 예시는 사용자가 직접 발급받은 키로 2026-08-21 실제 호출·수집한 데이터에서 인증키를 제외하고 발췌했다.